

Java Inheritance Programs

Execution Steps, Algorithm, Source Code and Output

This PDF contains four corrected Java inheritance programs: Single, Multilevel, Hybrid, and Hierarchical Inheritance.

1. Single Inheritance

1) Execution Steps

- Create a Java package named inheritance.
- Create the parent class One with variable x and method show().
- Create class Two using extends One so that it inherits x and show().
- Create an object t of class Two.
- Call t.see() to display the parent and child class variables.
- Call t.show() to execute the inherited method.

2) Algorithm

- Start.
- Define class One with x = 27 and show().
- Define class Two extends One and declare y = 34.
- Define see() using super.x and this.y.
- Create object t of Two.
- Call see() and show().
- Stop.

3) Source Code

```
package inheritance;

class One {
    int x = 27;

    void show() {
        System.out.println("This is One");
    }
}

class Two extends One {
    int y = 34;

    void see() {
        System.out.println("see: " + super.x + " " + this.y);
    }
}

public class Single {
    public static void main(String[] args) {
        Two t = new Two();
```

```
        t.see();  
        t.show();  
    }  
}
```

4) Output

```
see: 27 34  
This is One
```

2. Multilevel Inheritance

1) Execution Steps

- Create class C1 as the parent class with x = 50.
- Create class C2 extends C1 and add y = 30.
- Create class C3 extends C2 and add z = 12.
- In demo(), access x and y using super and z using this.
- Create an object of C3.
- Call demo() and addition().

2) Algorithm

- Start.
- Define C1 with x = 50.
- Define C2 extends C1 with y = 30.
- Define C3 extends C2 with z = 12.
- Display x, y and z in demo().
- Calculate x + y + z in addition().
- Create C3 object and call both methods.
- Stop.

3) Source Code

```
package inheritance;

class C1 {
    int x = 50;

    void show() {
        System.out.println("This is C1");
    }
}

class C2 extends C1 {
    int y = 30;

    void see() {
        System.out.println("see: " + super.x + " " + this.y);
    }
}

class C3 extends C2 {
    int z = 12;

    void demo() {
        System.out.println("see: " + super.x + " " + super.y + " " + this.z);
    }

    void addition() {
        System.out.println("addition: " + (x + y + z));
    }
}

public class Multilevel {
```

```
        public static void main(String[] args) {  
            C3 T = new C3();  
            T.demo();  
            T.addition();  
        }  
    }
```

4) Output

```
see: 50 30 12  
addition: 92
```

3. Hybrid Inheritance

1) Execution Steps

- Create parent class p1 with x = 27 and show().
- Create p2 extends p1 with y = 34.
- Create p3 extends p2 with z = 12 and demo().
- Create p4 extends p1 with z = 12 and demos().
- Create an object of p3 and call demo().
- Create an object of p4 and call demos().

2) Algorithm

- Start.
- Define p1 as the base class.
- Define p2 extending p1.
- Define p3 extending p2.
- Define p4 extending p1.
- Create p3 and p4 objects.
- Display inherited and local variables using super and this.
- Stop.

3) Source Code

```
package inheritance;

class p1 {
    int x = 27;

    void show() {
        System.out.println("This is One");
    }
}

class p2 extends p1 {
    int y = 34;

    void see() {
        System.out.println("see: " + super.x + " " + this.y);
    }
}

class p3 extends p2 {
    int z = 12;

    void demo() {
        System.out.println("see: " + super.x + " " + super.y + " " + this.z);
    }
}

class p4 extends p1 {
    int z = 12;

    void demos() {
        System.out.println("see: " + super.x + " " + this.z);
    }
}
```

```
    }  
}  
  
public class Hybrid {  
    public static void main(String[] args) {  
        p3 p = new p3();  
        p.demo();  
  
        p4 s = new p4();  
        s.demos();  
    }  
}
```

4) Output

```
see: 27 34 12  
see: 27 12
```

4. Hierarchical Inheritance

1) Execution Steps

- Create parent class add with x = 27.
- Create class sub extends add with y = 34.
- Create class smart extends add with z = 12.
- Create an object u of sub and an object s of smart.
- Call u.see() to display x and y.
- Call s.demo() to display x and z.

2) Algorithm

- Start.
- Define parent class add with x = 27.
- Define sub extends add with y = 34.
- Define smart extends add with z = 12.
- Create objects of sub and smart.
- Call see() and demo().
- Display the results.
- Stop.

3) Source Code

```
package inheritance;

class add {
    int x = 27;

    void show() {
        System.out.println("This is One");
    }
}

class sub extends add {
    int y = 34;

    void see() {
        System.out.println("see: " + super.x + " " + this.y);
    }
}

class smart extends add {
    int z = 12;

    void demo() {
        System.out.println("see: " + super.x + " " + this.z);
    }
}

public class Hierarchical {
    public static void main(String[] args) {
        sub u = new sub();
        smart s = new smart();
        u.see();
    }
}
```

```
        s.demo();  
    }  
}
```

4) Output

```
see: 27 34  
see: 27 12
```